ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ИМЕНИ ПАТРИСА ЛУМУМБЫ

# Отчет
о выполнении лабораторной работы № 4
"Глубокие сети прямого распространения"
по дисциплине "Нейронные сети"

Вариант задания № 13

Выполнил студент группы
ЗФИбд-01-24
Князев А. М. _____

Проверил и принял
Леонов С. С. _____

с оценкой _____

Москва, 2026

**Цель работы:** изучение глубоких полносвязных нейронных сетей прямого распространения и их применение в задачах бинарной классификации, многоклассовой классификации и регрессии.

**Основные этапы работы:**
1. Построить бинарный классификатор эмоциональной окраски отзывов из набора данных IMDB.
2. Построить многоклассовый классификатор новостных лент из набора данных Reuters.
3. Построить регрессионную модель прогноза цен на дома (Boston Housing) с проверкой по K блокам.

**Программное обеспечение:** Python 3.x, Jupyter Notebook, библиотеки PyTorch, NumPy и Matplotlib.

**Сценарий выполнения работы**

**Этап 1. Бинарная классификация отзывов IMDB**

Набор данных IMDB загружается из локального архива, тексты ограничены 700 отзывами на класс. Каждый отзыв преобразуется в вектор размерности 5000 по схеме мульти-горячего кодирования (наличие слова из словаря). Данные разделены на обучающее, контрольное и тестовое подмножества (1792 / 448 / 560 примеров).

Согласно варианту 13 сеть содержит три скрытых слоя по 16, 32 и 16 нейронов с функциями активации relu, relu и tanh; выходной слой — один нейрон с сигмоидой. В качестве функции потерь по варианту используется среднеквадратическая погрешность (MSE), оптимизатор — Adam. Результаты сопоставляются с базовым классификатором (2 слоя по 16 нейронов, relu-relu, бинарная перекрёстная энтропия).

Таблица 1 — Структура сети (этап 1, вариант 13)

| Параметр | Значение |
|---|---|
| Размер входного вектора | 5000 |
| Скрытые слои | 16 → 32 → 16 |
| Функции активации | relu, relu, tanh |
| Выходной слой | 1 нейрон, sigmoid |
| Функция потерь | MSE (среднеквадратическая погрешность) |
| Оптимизатор | Adam |

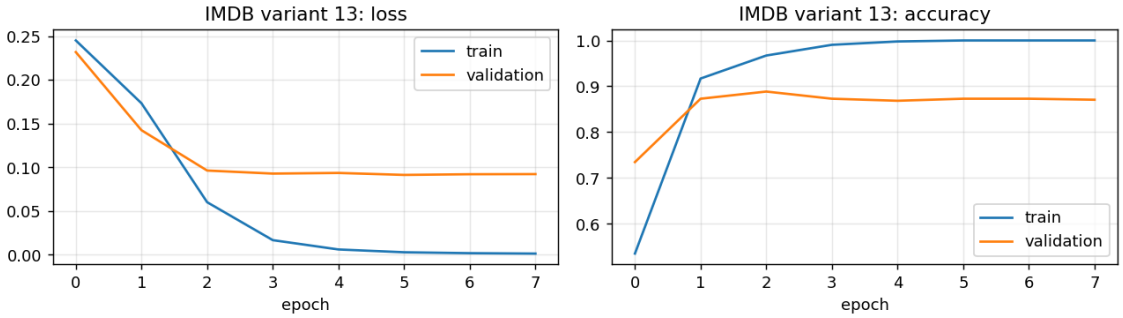



Рис. 1 — Графики функции потерь и точности (IMDB, вариант 13)

Таблица 2 — Результаты на тестовом подмножестве (этап 1)

| Модель | Test loss | Test accuracy |
|---|---|---|
| Вариант 13 (16-32-16, MSE) | 0.0769 | 0.8964 |
| Базовая (16-16, BCE) | 0.2458 | 0.8946 |

Обе модели достигают сопоставимой точности около 0.89; конфигурация варианта 13 даёт незначительно более высокий результат при использовании заданной функции потерь.

## Этап 2. Многоклассовая классификация Reuters

Набор данных Reuters содержит новостные ленты, распределённые по 46 классам. Тексты кодируются мульти-горячим вектором, метки — индексами классов. Данные разделены на обучающее, контрольное и тестовое подмножества (7185 / 1797 / 2246 примеров).

Согласно варианту 13 сеть содержит три скрытых слоя по 32, 64 и 64 нейрона с функциями активации tanh, mish и relu; выходной слой — 46 нейронов с softmax. Функция потерь — многокатегориальная перекрёстная энтропия, оптимизатор — Adam. Базовая модель — два слоя по 64 нейрона (relu-relu).

Таблица 3 — Структура сети (этап 2, вариант 13)

| Параметр | Значение |
|---|---|
| Число классов | 46 |
| Скрытые слои | 32 → 64 → 64 |
| Функции активации | tanh, mish, relu |
| Выходной слой | 46 нейронов, softmax |
| Функция потерь | категориальная перекрёстная энтропия |

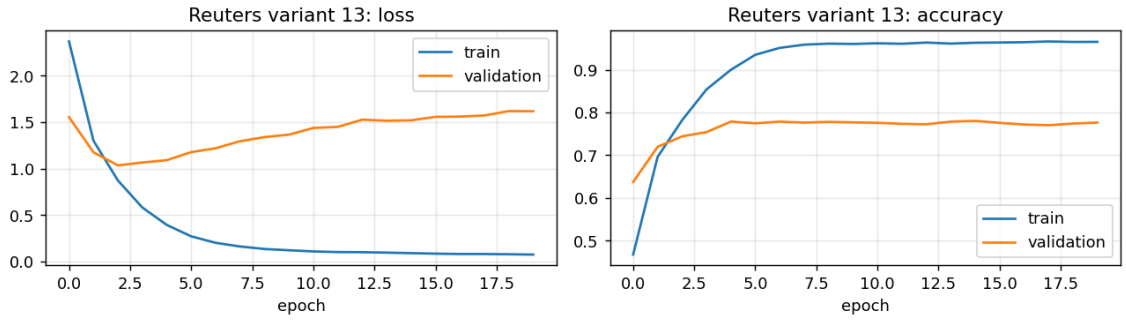



Рис. 2 — Графики функции потерь и точности (Reuters, вариант 13)

Таблица 4 — Результаты на тестовом подмножестве (этап 2)

| Модель | Test loss | Test accuracy |
|---|---|---|
| Вариант 13 (32-64- | 1.7665 | 0.7467 |

| | | |
|---|---|---|
| 64) | | |
| Базовая (64-64) | 1.2476 | 0.7796 |

Базовая конфигурация показала несколько более высокую точность. Это объясняется заданным вариантом сочетанием функций активации и относительно небольшим числом нейронов в первом скрытом слое — гиперпараметры варианта не являются оптимальными для данной задачи.

**Этап 3. Регрессия Boston Housing с перекрёстной проверкой по К блокам**

Набор данных Boston Housing содержит 506 примеров с 13 признаками. Признаки стандартизируются. Поскольку вариант 13 предполагает три скрытых слоя, число блоков перекрёстной проверки выбрано К = 4 (число слоёв плюс один).

Согласно варианту 13 сеть содержит три скрытых слоя по 32, 64 и 64 нейрона с функциями активации tanh, mish и relu; выходной слой — один нейрон без активации. Функция потерь — среднеквадратическая погрешность, метрика качества — средняя абсолютная погрешность (MAE), оптимизатор — Adam. Базовая модель — два слоя по 64 нейрона (relu-relu).

Таблица 5 — Структура сети (этап 3, вариант 13)

| Параметр | Значение |
|---|---|
| Число признаков | 13 |
| Скрытые слои | 32 → 64 → 64 |
| Функции активации | tanh, mish, relu |
| Выходной слой | 1 нейрон, линейный |
| Функция потерь / метрика | MSE / MAE |
| Число блоков К | 4 |

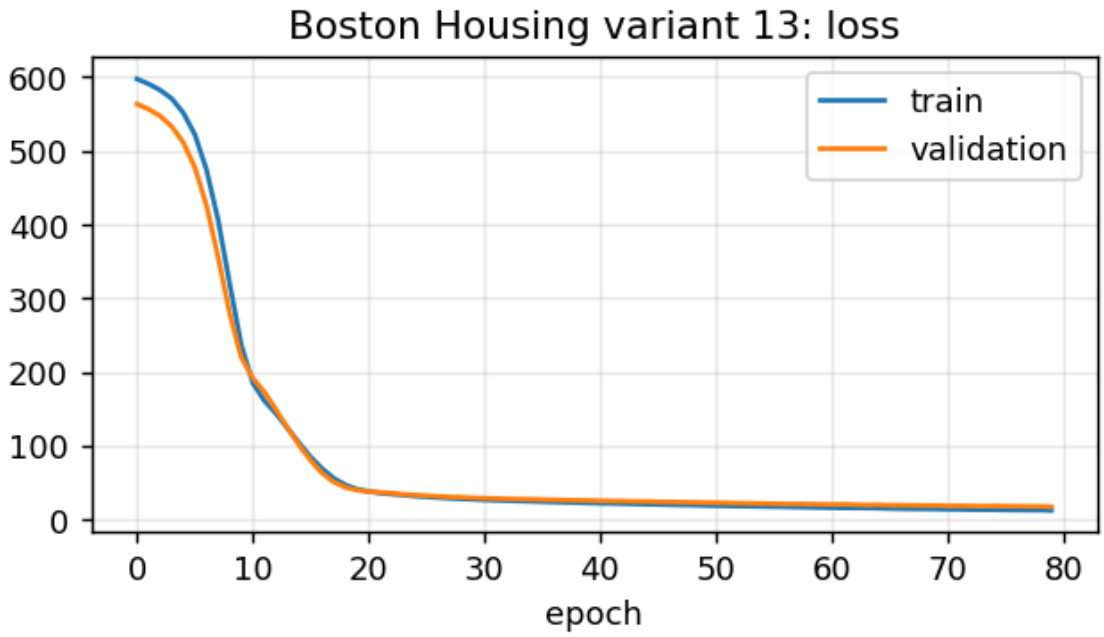



Рис. 3 — График функции потерь при обучении (Boston Housing, вариант 13)

Таблица 6 — Результаты перекрёстной проверки (MAE по блокам)

| Модель | MAE по блокам | Средняя MAE |
|---|---|---|
| Вариант 13 (32-64-64) | [2.82, 2.64, 3.28, 3.01] | 2.94 |
| Базовая (64-64) | [2.89, 2.75, 2.82, 2.62] | 2.77 |

Средняя абсолютная погрешность прогноза цены составляет около 2.9 тыс. долларов, что является приемлемым результатом для данного набора. Модели варианта 13 и базовой конфигурации показывают близкие значения MAE.

## Код программы

```
import random
import zipfile
from pathlib import Path


import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import KFold, train_test_split
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset


DEVICE = torch.device(
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)
SEED = 13
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)



def existing_path(*variants: str) -> Path:
    for variant in variants:
        path = Path(variant)
        if path.exists():
            return path
    return Path(variants[0])



def local_dataset_file(filename: str) -> Path:
    path = existing_path(f"data/{filename}", f"../data/{filename}")
    if not path.exists():
        raise FileNotFoundError(
            f"Не найден {filename}. Скачай его один раз в папку data
рядом с ноутбуком."
        )
    return path



print(f"Device: {DEVICE}")
```

```
def activation(name: str) -> nn.Module:
    activations = {
        "relu": nn.ReLU(),
        "tanh": nn.Tanh(),
        "mish": nn.Mish(),
        "elu": nn.ELU(),
        "leaky_relu": nn.LeakyReLU(0.1),
    }
    return activations[name]



def make_mlp(input_size: int, hidden: list[int], activations:
list[str], output_size: int) -> nn.Sequential:
    layers = []
    prev = input_size
    for width, act in zip(hidden, activations):
        layers.append(nn.Linear(prev, width))
        layers.append(activation(act))
        prev = width
    layers.append(nn.Linear(prev, output_size))
    return nn.Sequential(*layers)



def make_loader(x: np.ndarray, y: np.ndarray, batch_size: int = 64,
shuffle: bool = True) -> DataLoader:
    dataset = TensorDataset(torch.tensor(x, dtype=torch.float32),
torch.tensor(y))
    return DataLoader(dataset, batch_size=batch_size, shuffle=shuffle)


def count_correct(logits: torch.Tensor, y: torch.Tensor, task: str) ->
int:
    if task == "regression":
        return 0
    if task == "multiclass":
        return int((logits.argmax(dim=1) == y).sum().item())
    if task == "binary_probability":
        pred = (logits.view(-1) >= 0.5).float()
    else:
        pred = (torch.sigmoid(logits).view(-1) >= 0.5).float()
    return int((pred == y.view(-1)).sum().item())



def evaluate(model: nn.Module, loader: DataLoader, criterion:
nn.Module, task: str) -> tuple[float, float]:
    model.eval()
```

```
    total_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            out = model(x)
            loss = criterion(out, y)
            total_loss += float(loss.item()) * len(x)
            correct += count_correct(out, y, task)
            total += len(x)
    accuracy = correct / total if task != "regression" else 0.0
    return total_loss / total, accuracy



def train_model(
    model: nn.Module,
    train_loader: DataLoader,
    val_loader: DataLoader,
    criterion: nn.Module,
    task: str,
    epochs: int = 10,
    lr: float = 1e-3,
) -> dict[str, list[float]]:
    model.to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    history = {"loss": [], "val_loss": [], "acc": [], "val_acc": []}


    for _ in range(epochs):
        model.train()
        train_loss = 0.0
        train_correct = 0
        train_total = 0
        for x, y in train_loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            optimizer.zero_grad()
            out = model(x)
            loss = criterion(out, y)
            loss.backward()
            optimizer.step()
            train_loss += float(loss.item()) * len(x)
            train_correct += count_correct(out, y, task)
            train_total += len(x)


        val_loss, val_acc = evaluate(model, val_loader, criterion,
task)
        history["loss"].append(train_loss / train_total)
```

```
        history["acc"].append(train_correct / train_total if task !=
"regression" else 0.0)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)

    return history

def plot_history(history: dict[str, list[float]], title: str,
show_accuracy: bool = True) -> None:
    cols = 2 if show_accuracy else 1
    fig, axes = plt.subplots(1, cols, figsize=(10 if show_accuracy else
5, 3))
    if not show_accuracy:
        axes = [axes]

    axes[0].plot(history["loss"], label="train")
    axes[0].plot(history["val_loss"], label="validation")
    axes[0].set_title(f"{title}: loss")
    axes[0].set_xlabel("epoch")
    axes[0].grid(True, alpha=0.3)
    axes[0].legend()

    if show_accuracy:
        axes[1].plot(history["acc"], label="train")
        axes[1].plot(history["val_acc"], label="validation")
        axes[1].set_title(f"{title}: accuracy")
        axes[1].set_xlabel("epoch")
        axes[1].grid(True, alpha=0.3)
        axes[1].legend()

    plt.tight_layout()
    plt.show()

def read_imdb(zip_path: Path, max_docs_per_class: int = 700) ->
tuple[list[str], np.ndarray]:
    if not zip_path.exists():
        raise FileNotFoundError(f"Не найден архив IMDB: {zip_path}")

    texts = []
    labels = []
    with zipfile.ZipFile(zip_path) as archive:
        for split in ["train", "test"]:
            for label_name, label in [("neg", 0), ("pos", 1)]:
                prefix = f"aclImdb/{split}/{label_name}/"
                names = [n for n in archive.namelist() if
n.startswith(prefix) and n.endswith(".txt")]
```

```
                names = [n for n in names if "/._" not in n]
                for name in names[:max_docs_per_class]:
                    texts.append(archive.read(name).decode("utf-8",
errors="ignore"))
                    labels.append(label)

    return texts, np.array(labels, dtype=np.float32)


texts, y = read_imdb(existing_path("IMDB/aclImdb.zip",
"../IMDB/aclImdb.zip"), max_docs_per_class=700)


x_train_text, x_test_text, y_train, y_test = train_test_split(
    texts, y, test_size=0.2, stratify=y, random_state=SEED
)
x_train_text, x_val_text, y_train, y_val = train_test_split(
    x_train_text, y_train, test_size=0.2, stratify=y_train,
random_state=SEED
)


vectorizer = CountVectorizer(max_features=5000, binary=True,
stop_words="english")
x_train =
vectorizer.fit_transform(x_train_text).astype("float32").toarray()
x_val = vectorizer.transform(x_val_text).astype("float32").toarray()
x_test = vectorizer.transform(x_test_text).astype("float32").toarray()


print(x_train.shape, x_val.shape, x_test.shape)


def column(values: np.ndarray) -> np.ndarray:
    return values.reshape(-1, 1).astype("float32")


imdb_model = nn.Sequential(
    make_mlp(x_train.shape[1], [16, 32, 16], ["relu", "relu", "tanh"],
1),
    nn.Sigmoid(),
)
imdb_baseline = make_mlp(x_train.shape[1], [16, 16], ["relu", "relu"],
1)


train_loader = make_loader(x_train, column(y_train), batch_size=64)
val_loader = make_loader(x_val, column(y_val), batch_size=128,
shuffle=False)
test_loader = make_loader(x_test, column(y_test), batch_size=128,
shuffle=False)
```

```
imdb_history = train_model(
    imdb_model, train_loader, val_loader, nn.MSELoss(),
task="binary_probability", epochs=8
)
baseline_history = train_model(
    imdb_baseline, train_loader, val_loader, nn.BCEWithLogitsLoss(),
task="binary", epochs=8
)


imdb_loss, imdb_acc = evaluate(imdb_model, test_loader, nn.MSELoss(),
"binary_probability")
base_loss, base_acc = evaluate(imdb_baseline, test_loader,
nn.BCEWithLogitsLoss(), "binary")


print(f"IMDB variant: test_loss={imdb_loss:.4f},
test_acc={imdb_acc:.4f}")
print(f"IMDB baseline: test_loss={base_loss:.4f},
test_acc={base_acc:.4f}")


plot_history(imdb_history, "IMDB variant 13")
plot_history(baseline_history, "IMDB baseline")


reuters_path = local_dataset_file("reuters.npz")


with np.load(reuters_path, allow_pickle=True) as data:
    reuters_sequences = data["x"]
    reuters_labels = data["y"].astype("int64")


# Повторяем стандартную схему keras.datasets.reuters.load_data:
перемешивание с seed=113
# и разделение 80/20 на train/test.
indices = np.arange(len(reuters_sequences))
rng = np.random.default_rng(113)
rng.shuffle(indices)
reuters_sequences = reuters_sequences[indices]
reuters_labels = reuters_labels[indices]


split = int(len(reuters_sequences) * 0.8)
x_train_seq = reuters_sequences[:split]
y_train_full = reuters_labels[:split]
x_test_seq = reuters_sequences[split:]
y_test = reuters_labels[split:]


NUM_WORDS = 10000
```

```
def vectorize_sequences(sequences, dimension: int = NUM_WORDS) ->
np.ndarray:
    result = np.zeros((len(sequences), dimension), dtype="float32")
    for row, sequence in enumerate(sequences):
        indexes = [index for index in sequence if index < dimension]
        result[row, indexes] = 1.0
    return result



x_train_full = vectorize_sequences(x_train_seq)
x_test = vectorize_sequences(x_test_seq)


x_train, x_val, y_train, y_val = train_test_split(
    x_train_full, y_train_full, test_size=0.2, stratify=y_train_full,
random_state=SEED
)


num_classes = int(max(y_train_full.max(), y_test.max()) + 1)
multi_model = make_mlp(x_train.shape[1], [32, 64, 64], ["tanh", "mish",
"relu"], num_classes)
multi_baseline = make_mlp(x_train.shape[1], [64, 64], ["relu", "relu"],
num_classes)
criterion = nn.CrossEntropyLoss()


multi_history = train_model(
    multi_model,
    make_loader(x_train, y_train, batch_size=64),
    make_loader(x_val, y_val, batch_size=128, shuffle=False),
    criterion,
    task="multiclass",
    epochs=20,
)
train_model(
    multi_baseline,
    make_loader(x_train, y_train, batch_size=64),
    make_loader(x_val, y_val, batch_size=128, shuffle=False),
    criterion,
    task="multiclass",
    epochs=20,
)


multi_loss, multi_acc = evaluate(multi_model, make_loader(x_test,
y_test, batch_size=128, shuffle=False), criterion, "multiclass")
base_loss, base_acc = evaluate(multi_baseline, make_loader(x_test,
y_test, batch_size=128, shuffle=False), criterion, "multiclass")
```

```
print(f"Reuters train/val/test:
{len(x_train)}/{len(x_val)}/{len(x_test)}")
print(f"Reuters classes: {num_classes}")
print(f"Multiclass variant: test_loss={multi_loss:.4f},
test_acc={multi_acc:.4f}")
print(f"Multiclass baseline: test_loss={base_loss:.4f},
test_acc={base_acc:.4f}")


plot_history(multi_history, "Reuters variant 13")


boston_path = local_dataset_file("boston_housing.npz")


with np.load(boston_path) as data:
    x = data["x"].astype("float32")
    y = data["y"].astype("float32").reshape(-1, 1)


x = StandardScaler().fit_transform(x).astype("float32")


kfold = KFold(n_splits=4, shuffle=True, random_state=SEED)
fold_mae = []
baseline_mae = []
first_fold_history = None


for fold, (train_idx, val_idx) in enumerate(kfold.split(x), start=1):
    reg_model = make_mlp(x.shape[1], [32, 64, 64], ["tanh", "mish",
"relu"], 1)
    reg_baseline = make_mlp(x.shape[1], [64, 64], ["relu", "relu"], 1)
    criterion = nn.MSELoss()


    train_loader = make_loader(x[train_idx], y[train_idx],
batch_size=32)
    val_loader = make_loader(x[val_idx], y[val_idx], batch_size=32,
shuffle=False)


    history = train_model(reg_model, train_loader, val_loader,
criterion, task="regression", epochs=80, lr=5e-4)
    train_model(reg_baseline, train_loader, val_loader, criterion,
task="regression", epochs=80, lr=5e-4)


    reg_model.eval()
    reg_baseline.eval()
    with torch.no_grad():
        val_tensor = torch.tensor(x[val_idx], dtype=torch.float32,
device=DEVICE)
```

```
        pred = reg_model(val_tensor).cpu().numpy()
        base_pred = reg_baseline(val_tensor).cpu().numpy()

    fold_mae.append(float(np.mean(np.abs(pred - y[val_idx]))))
    baseline_mae.append(float(np.mean(np.abs(base_pred - y[val_idx]))))

    if first_fold_history is None:
        first_fold_history = history

print(f"Boston Housing samples/features: {x.shape[0]}/{x.shape[1]}")
print(f"Regression KFold MAE: {np.round(fold_mae, 2).tolist()},
mean={np.mean(fold_mae):.2f}")
print(f"Regression baseline MAE: {np.round(baseline_mae, 2).tolist()},
mean={np.mean(baseline_mae):.2f}")

plot_history(first_fold_history, "Boston Housing variant 13",
show_accuracy=False)
```

**Вывод**

В ходе работы построены и обучены три глубокие полносвязные сети прямого распространения для задач бинарной классификации (IMDB), многоклассовой классификации (Reuters) и регрессии (Boston Housing), а их результаты сопоставлены с базовыми моделями.

В задаче бинарной классификации IMDB сеть варианта 13 (16-32-16) достигла точности 0.8964 на тестовом подмножестве, незначительно превзойдя базовую модель.

В задаче многоклассовой классификации Reuters сеть варианта 13 (32-64-64) достигла точности 0.7467. Базовая модель (64-64) показала близкий результат 0.7796; разница объясняется заданным вариантом набором гиперпараметров.

В задаче регрессии Boston Housing средняя абсолютная погрешность по четырём блокам перекрёстной проверки составила 2.94, что сопоставимо с базовой моделью (2.77). Все этапы работы выполнены, получены графики функции потерь и метрик качества, проведено сопоставление с базовыми классификаторами.